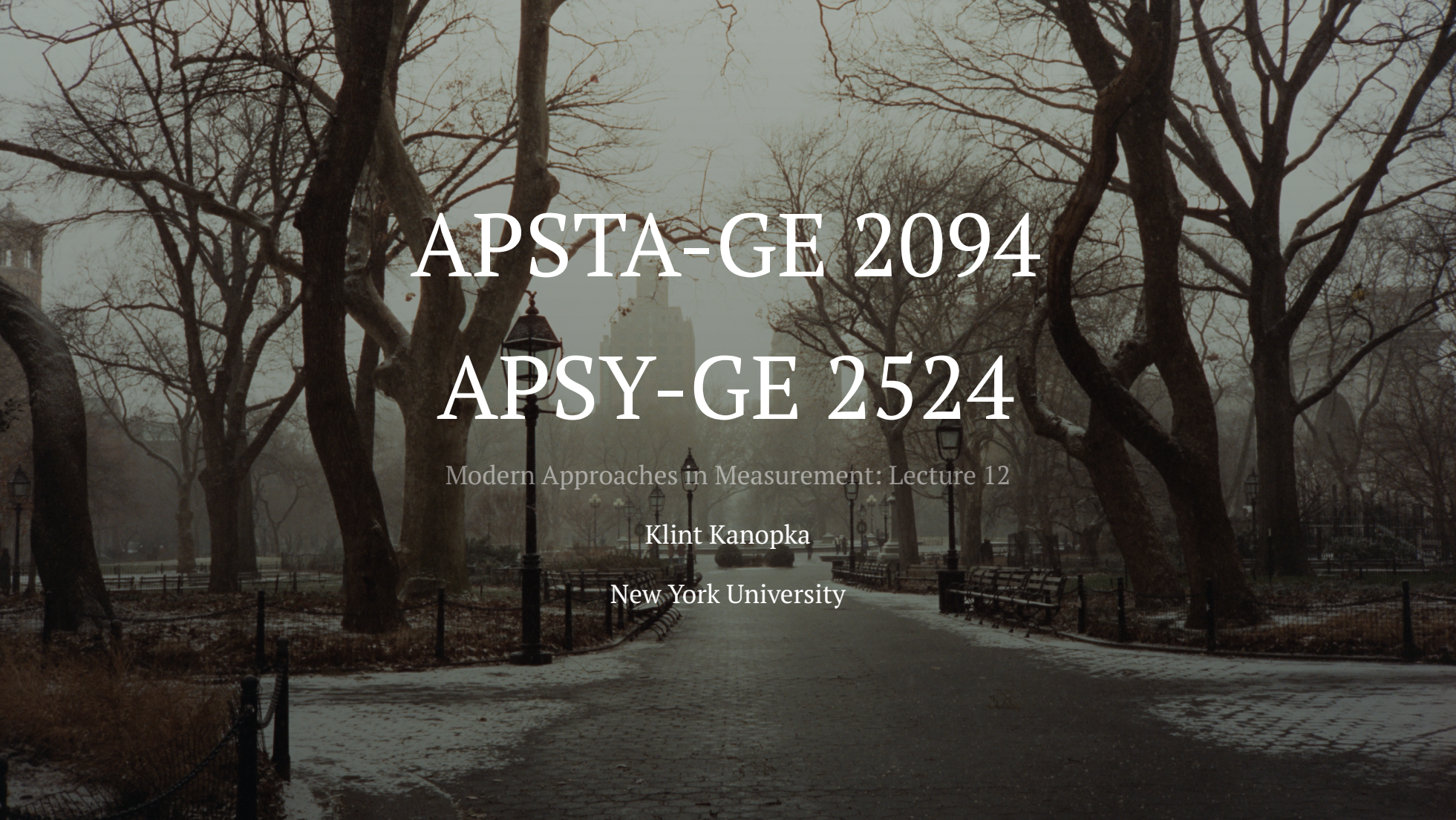




APSTA-GE 2094

APSY-GE 2524

Modern Approaches in Measurement: Lecture 12

Klint Kanopka

New York University

Table of Contents

1. Measurement - Week 12
 1. Table of Contents
2. Recommendations
 1. Formulating the Problem
 2. The Utility Matrix
 3. Movie Recommendation
 4. Approach One: Content-based Recommendations
 5. Approach Two: Collaborative Filtering
3. Latent Variable Models for Recommendation
 1. Latent Variable Models for Recommendation
4. Dating Apps
5. Wrap Up
 1. Recap

Announcements

- For PS6:
 - I didn't release it. It doesn't exist. Don't try to do it.
- For PS7:
 - It's worth 200 pts (twice a normal problem set)
 - This makes sense because it's more work than usual
 - I'd prefer you did a good job at this instead of jumped through normal pset hoops
 - It's due May 7 @ 11.59p
 - I can't take late submissions here because of grading deadlines!

Recommendations

Recommendations

- How do you learn about new stuff?
 - Recommendations!
 - Broadly we can think about two types of recommendations:
 - Generic
 - Personalized
- Generic recommendations don't care about who you are
 - Wirecutter articles
 - Curated lists from influencers
 - Best selling
 - Most popular
 - Trending
 - Recent uploads

Recommendation

- Personalized recommendations are, well, *personalized*
 - Suggestions from friends
 - Algorithmic recommendations

Formulating the Problem

- Recommendations require three parts:
 - X is the set of *users* (customers, viewers, etc.)
 - S is the set of *items* (products, posts, videos, songs, etc.)
 - The *utility function* $u : X \times S \rightarrow R$
 - R is a set of ratings
 - R is *totally ordered*, meaning any two elements can be compared
 - During comparisons, one item is either rated higher than the other or they are equally rated
 - Stars, thumbs up/down, swiping left/right, etc.

The Utility Matrix

- Ratings are stored in an object called the *utility matrix*
- The problem is that every user doesn't rate every item
 - The utility matrix is *sparse*

	Taylor Swift	Bad Bunny	Turnstile	Pantera	Crowbar
Alice	5	3	3		
Bob	1			5	
Carol	4		5		
David			4	4	5

- Does this remind you of anything we've dealt with before?

The Two Core Problems of Recommendation

- Problem One: Gathering observed ratings
 - This is the data collection problem
 - Where might this information come from?
 - Explicit ratings
 - Ask people to rate items
 - Pay people to rate items
 - Implicit ratings
 - Learn ratings from user behavior
- Problem Two: Predicting unobserved ratings
 - Upside is that you only really care about predicting high unknown ratings!
 - Downside is that this is a tricky problem and measuring success isn't always straightforward
 - Any early ideas on how this can be done?

Movie Recommendation

- With a group of 3 ± 1 :

1. Download the MovieLens 100K Dataset
2. Unzip the files, check out the `README` to get a sense of what's in there and how the data are organized
3. The full dataset is in `u.data`. Using whatever you like from the files provided, come up with a plan for recommending new movies to users
4. Try to code up a couple of test cases or mild proof-of-concepts
5. Start with something very simple and then add complexity!

```
1 d <- read_delim('u.data', col_names=c('user', 'movie', 'rating', 'timestamp'))
```

Approach One: Content-based Recommendations

- **Core idea:** Items have features and you recommend new items to a user that have features similar to items they've already rated highly
 - Restaurants that have the same owner or head chef
 - If you like Thai Diner, you should try Mommy Pai's
 - Bands that share members
 - If you like Pantera and Crowbar, you should listen to Down
 - Movies by the same director
 - If you like Raiders of the Lost Ark and Hook, you should watch Schindler's List
- How do you decide on the features?
 - Hand pick them
- How do you decide on similarity?
 - Use a distance metric!

Approach One: Content-based Recommendations

- Pros:
 - Independent of other users
 - You can handle users with niche tastes
 - You can recommend new and unpopular items
 - Recommendations are explainable
- Cons:
 - Feature engineering is hard
 - Recommending to new users doesn't really work
 - Limits the diversity of things you show to a user

Approach Two: Collaborative Filtering

- **Core idea:** We find users who have rated things similarly to you and then recommend to you things they've liked
 - We use the observed ratings of an item as its feature set
 - We use the observed ratings of a user as its feature set
 - Recommendations are made based on matching similar users (user-user collaborative filtering) or matching similar items (item-item collaborative filtering)
- How might this work in practice?

User-User Collaborative Filtering

- For a user $x \in X$:
 - Find the set of K other users who have ratings most similar to x
 - Estimate x 's ratings on unseen items using the ratings of those K users
- How might you determine similarity of ratings?
 - Cosine similarity, Jaccard similarity, correlations, lots of options!
- How would you then estimate x 's rating on a new item?
 - You can take the average of other users' ratings
 - You can take a weighted average using their similarity to x

Item-Item Collaborative Filtering

- For a user x and item $s \in S$:
 - Find the set of K other items who have ratings most similar to x
 - Estimate x 's rating of s using the ratings of those K items
- This generally boils down to k -Nearest Neighbors (kNN) followed by a weighted average
- Which version do you think works better? Why?
 - Item-item usually works better, as items are simpler

Approach Two: Collaborative Filtering

- Pros:
 - Works for any kinds of items
 - Does not require feature selection
- Cons:
 - Cold start problems
 - You need a critical mass of users and ratings
 - Sparsity problem
 - Might be hard to find users who have rated similar items
 - Unable to recommend items that have never been rated
 - Popularity bias
 - Users with unique tastes don't get good recommendations
 - Tends to over recommend popular items

Latent Variable Models for Recommendation

Latent Variable Models for Recommendation

- What tools do we have that might be helpful here?
- Something like PCA or Factor Analysis is great for finding latent dimensions of preference
 - You have to be careful about missing data
 - Remember that factor analysis can be run on a *pairwise complete* correlation matrix!
- Clustering can provide us with groups of users or items with similar profiles
- Multidimensional IRT is actually a *great* tool for this job!
 - Using a polytomous model, you can get predicted probabilities of ratings that aren't just dichotomous choices
- Try applying a latent variable approach to the MovieLens data, what do you learn?
 - Do this now!

Dating Apps

Dating Apps

- I think most people have tried dating apps?
- A rough dating app typology:
 - Question-based
 - OKCupid and older apps
 - Swipe-based
 - Tinder, Hinge, etc
 - Location-based
 - Grindr, Scruff
- Other stuff?

Dating Apps

- What's the goal of a dating app?
 - Make money
 - Sell premium features
 - Maximize ad exposure
 - Connect people
 - If people don't like who they see, they won't use the app
 - If nobody talks to a person, they won't use the app
- What could we do if we design our own app?
 - How would you design the app?
 - How would you match users?
 - Talk to a few neighbors about it!

Tinder

- Tinder used to use an Elo system (Bumble, too)
- The probability person i swipes right on person j , $X_{ij} = 1$, depends on the difference between their- latent scores in the app
 - Assumes people will swipe left ($X_{ij} = 0$) on people they view as “beneath them”
 - Goal: Display profiles to maximize the probability that they swipe right *on each other*
 - When is this probability maximized?
- Turns out this made people really upset
- Now they display people based on app usage

Hinge

- Hinge is “designed to be deleted” or something
- They treat dating as a stable matching problem
 - Assume we have two groups that attempt to match with each other, A , B
 - Pairings are not stable if a member of A prefers some member of B over the one they're matched with
 - *and* the member of B they prefer also prefers them over who the member of A they're matched with
- The solution to this is the Gale-Shapley Algorithm (which won the 2012 Nobel Prize in Economics)
 - This requires group members to develop rankings of members of the other group

Gale-Shapley Algorithm

- How it works:
 - In each iteration, members of group A who have not yet matched make an offer to their top choice in group B who they have not yet made an offer to
 - Members of group B evaluate their offers against their current match and either accept the best new offer or reject all offers
 - Repeat until all members are matched
- Guaranteed to match everyone
- Guaranteed to produce stable matches

Back to Hinge

- What's the problem?
- Users don't produce rankings of everyone they're trying to date!
- How does Hinge infer rankings?
 - They use interaction patterns to determine the type of users you prefer
 - They're not specific about the methods they use, but we could make some guesses!
 - How would you do it?

OKCupid

- Individuals respond to questions and describe how their ideal partner would respond to those questions
 - Additionally they specify how important responses are to them
- OKCupid provides users with “Match %”
 - They’re not super specific on what the actual model is
- How could we do this?

OKCupid Experiments

- OKCupid has been really open about the fact that they collect lots of data and experiment on their users
- The book *Dataclysm* is a super interesting documentation of applied statistics, measurement, and data science from the founder of OKCupid
- In 2013 OKCupid made a bunch of people pretty upset:
 - They basically just randomly rearranged the match scores
 - Turns out, if you tell people they're a high percentage match, they're more likely to talk to each other and keep talking
- The OKCupid Data Blog used to be full of cool stuff, but it's really only accessible via the Wayback Machine

Wrap Up

Recap

- Latent variable models and unsupervised learning are used to perform tasks beyond just measurement
- Commonly used in recommender systems
 - Generally they outperform pure collaborative filtering approaches
- Dating apps are another place that have used latent variable models
- Remember: PS6 no longer exists
- If you have questions or need support on PS7, I'm available!